



**TECNOLÓGICO
NACIONAL DE MÉXICO**



Ing. En Sistemas Computacionales

Tópicos de IA

10:00 – 11:00

Bonus 2, N Reinas

Alumnas:

Gavilanes Medina Tania Elizabeth

Pacheco Reyes Kimberlyn

Ramirez Pompa Marian Gabriela

Maestro: Zuriel Dathan Mora Felix

Explicación

Descripción:

El problema de las N reinas consiste en colocar N reinas en un tablero de ajedrez de $N \times N$ casillas de tal manera que ninguna reina pueda atacar a otra.

En ajedrez, una reina ataca en la misma fila, columna o diagonal.

Con el objetivo de encontrar una configuración donde todas las reinas estén en posiciones seguras (sin colisiones).

Complejidad:

Para $N=8$, existen 92 soluciones distintas. El espacio de búsqueda es de $N!$ posibles configuraciones ($8! = 40320$ para $N=8$).

Estrategia de solución:

Se utiliza Búsqueda Tabú, una metaheurística que explora el espacio de soluciones evitando quedar atrapada en óptimos locales mediante una lista de movimientos prohibidos temporalmente (lista tabú).

Representación del P y S (Problema/Solución)

ESTADO (Solución):

- Se representa como un arreglo de tamaño N
- Índice i ($0..N-1$): representa la columna $i+1$
- Valor en posición i : representa la fila donde está la reina ($1..N$)
- Esta representación garantiza que no hay dos reinas en la misma columna

Ejemplo para $N=8$:

Solución = [4, 2, 7, 3, 6, 8, 5, 1]

Interpretación:

Columna 1: Reina en fila 4

Columna 2: Reina en fila 2

Columna 3: Reina en fila 7

Columna 4: Reina en fila 3

Columna 5: Reina en fila 6

Columna 6: Reina en fila 8

Columna 7: Reina en fila 5

Columna 8: Reina en fila 1

Función Objetivo (f):

$f(s)$ = número de pares de reinas que se atacan entre sí en diagonales

Solución óptima: $f(s) = 0$

Operador:

Intercambiar los valores en dos posiciones i y j del arreglo. Equivale a mover dos reinas de columna

Pseudocodigo

ALGORITMO N_REINAS_TABU

// Resuelve el problema de las N reinas usando búsqueda tabú

// Entrada: N (tamaño del tablero)

// Salida: Solución con posiciones de las reinas

INICIO

// Función principal

FUNCION principal()

 ESCRIBIR "PROBLEMA DE LAS N REINAS"

 ESCRIBIR "Ingrese el tamaño del tablero: "

LEER N

SI $N < 4$ ENTONCES

 ESCRIBIR "Para $N < 4$ no hay solución (excepto $N=1$)."

 RETORNAR

FIN SI

ESCRIBIR "Iniciando búsqueda para $N=$ ", N, "..."

// Llamar a la búsqueda tabú

(solucion, costo, metricas) = busqueda_tabu(N)

// Mostrar y guardar informe

mostrar_e_guardar_informe(N, solucion, costo, metricas)

FIN FUNCION

// Genera una solución inicial aleatoria

FUNCION generar_solucion_aleatoria(N)

 // Crea una permutación aleatoria de $[1..N]$

 vector solucion $[1..N]$

 PARA i DESDE 1 HASTA N

 solucion[i] = i

 FIN PARA

// Mezclar aleatoriamente

PARA i DESDE N HASTA 2 PASO -1

```

    j = aleatorio(1, i)
    intercambiar(solucion[i], solucion[j])
FIN PARA

RETORNAR solucion
FIN FUNCION

// Calcula el número de colisiones en una solución
FUNCION calcular_colisiones(solucion, N)
    // Arreglos para diagonales
    diag1[0..2*N] // Diagonales principales
    diag2[0..2*N-1] // Diagonales secundarias

    // Inicializar en 0
    PARA i DESDE 0 HASTA 2*N
        diag1[i] = 0
        diag2[i] = 0
    FIN PARA

    colisiones = 0

    PARA i DESDE 1 HASTA N
        // Calcular diagonales para la reina en (i, solucion[i])
        d1 = i + solucion[i]    // Diagonal principal
        d2 = i - solucion[i] + N // Diagonal secundaria

```

```
// Sumar colisiones existentes en estas diagonales
colisiones = colisiones + diag1[d1] + diag2[d2]

// Incrementar contadores de diagonales
diag1[d1] = diag1[d1] + 1
diag2[d2] = diag2[d2] + 1
FIN PARA

RETORNAR colisiones
FIN FUNCION
```

```
// Genera un vecino intercambiando dos posiciones
FUNCION intercambiar(solucion, i, j)
    vector vecino = copia(solucion)
    intercambiar(vecino[i], vecino[j])
    RETORNAR vecino
FIN FUNCION
```

```
// Imprime el tablero en formato gráfico
FUNCION imprimir_tablero(solucion, N)
    ESCRIBIR línea de separación

    PARA fila DESDE 1 HASTA N
        PARA columna DESDE 1 HASTA N
            SI solucion[columna] == fila ENTONCES
```

```

        ESCRIBIR " Q"
    SINO
        ESCRIBIR " ."
    FIN SI
FIN PARA
    ESCRIBIR nueva línea
FIN PARA

    ESCRIBIR línea de separación
FIN FUNCION

// Algoritmo principal de búsqueda tabú
FUNCION busqueda_tabu(N, max_iter_sin_mejora, max_iteraciones)
    tiempo_inicio = tiempo_actual()

    // Inicialización
    solucion_actual = generar_solucion_aleatoria(N)
    costo_actual = calcular_colisiones(solucion_actual, N)

    mejor_solucion = copia(solucion_actual)
    mejor_costo = costo_actual
    iteraciones = 0
    steps = vector_vacio()

    lista_tabu = cola_vacia()
    iter_sin_mejora = 0

```

```

// Si ya es solución óptima, retornar
SI costo_actual == 0 ENTONCES
    RETORNAR mejor_solucion, mejor_costo, metricas
FIN SI

// Bucle principal
MIENTRAS iter_sin_mejora < max_iter_sin_mejora Y iteraciones < max_iteraciones
HACER
    mejor_vecino = NULO
    mejor_mov = NULO

    // Explorar todos los vecinos mediante intercambios
    PARA i DESDE 1 HASTA N-1
        PARA j DESDE i+1 HASTA N
            // Verificar si el movimiento está tabú
            mov_tabu = (i, j) EN lista_tabu

            vecino = intercambiar(solucion_actual, i, j)
            costo_vecino = calcular_colisiones(vecino, N)

            // Criterio de aspiración: si es tabú pero mejora mejor solución
            SI mov_tabu Y costo_vecino >= mejor_costo ENTONCES
                CONTINUAR // Saltar este vecino
            FIN SI

        // Seleccionar mejor vecino

```


SI mejor_vecino ES NULO O costo_vecino <
calcular_colisiones(mejor_vecino, N) ENTONCES

mejor_vecino = vecino

mejor_mov = (i, j)

FIN SI

FIN PARA

FIN PARA

// Si no se encontró vecino, terminar

SI mejor_vecino ES NULO ENTONCES

ROMPER

FIN SI

// Actualizar solución actual

solucion_actual = mejor_vecino

costo_actual = calcular_colisiones(mejor_vecino, N)

iteraciones = iteraciones + 1

// Actualizar mejor solución encontrada

SI costo_actual < mejor_costo ENTONCES

mejor_solucion = copia(solucion_actual)

mejor_costo = costo_actual

iter_sin_mejora = 0

SI mejor_costo == 0 ENTONCES

ROMPER // Solución óptima encontrada

FIN SI

SINO

```
iter_sin_mejora = iter_sin_mejora + 1
```

```
FIN SI
```

```
// Registrar movimiento en historial
```

```
AGREGAR mejor_mov A steps
```

```
// Actualizar lista tabú
```

```
AGREGAR mejor_mov A lista_tabu
```

```
SI longitud(lista_tabu) > (N // 2) + 1 ENTONCES
```

```
    ELIMINAR PRIMERO DE lista_tabu
```

```
FIN SI
```

```
FIN MIENTRAS
```

```
// Preparar métricas
```

```
metricas = {
```

```
    "iteraciones": iteraciones,
```

```
    "tiempo": tiempo_actual() - tiempo_inicio,
```

```
    "steps": steps
```

```
}
```

```
RETORNAR mejor_solucion, mejor_costo, metricas
```

```
FIN FUNCION
```

```
// Muestra y guarda el informe de resultados
```

```
FUNCION mostrar_e_guardar_informe(N, solucion, costo, metricas)
```

```
    resultado = "ÉXITO"
```

SI costo > 0 ENTONCES

 resultado = "NO ÓPTIMO (" + costo + " colisiones)"

FIN SI

informe = "PROBLEMA DE LAS " + N + " REINAS"

informe = informe + "\n" + "=" * 70

informe = informe + "\nSolución: " + solucion

informe = informe + "\nResultado: " + resultado

informe = informe + "\nTiempo: " + metricas.tiempo + "s"

informe = informe + "\nIteraciones: " + metricas.iteraciones

informe = informe + "\nPasos: " + longitud(metricas.steps) + " - " + metricas.steps

ESCRIBIR informe

imprimir_tablero(solucion, N)

// Guardar en archivo

archivo = ABRIR("informe_n_reinas.txt", "escritura")

ESCRIBIR archivo, informe

CERRAR archivo

FIN FUNCION

FIN ALGORITMO